

Laboratory 01

Toolchain & Environment Verification

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 150 MHz PLL)
RTOS	FreeRTOS via NXP MCUXpresso SDK (freertos-kernel-upstream)
Toolchain	ARM GNU Toolchain 14.2 (arm-none-eabi-gcc) + Make
Flash / Debug	pyocd 0.44 via local .venv (target mcxn236vdf)
Estimated time	2–3 hours
CLO mapping	CLO 1 (foundations) CLO 3 (FreeRTOS implementation)

1. Objectives

By completing this lab you will be able to:

1. Build a FreeRTOS project using the NXP MCUXpresso SDK and ARM cross-compiler.
2. Flash firmware and monitor the board using pyocd.
3. Verify that FreeRTOS creates and schedules tasks correctly via UART output and a physical LED.
4. Observe how task priority affects scheduling — the foundation for Module 2.
5. Identify and prevent a race condition on a shared resource using a mutex.

2. Prerequisites

Before starting, confirm these are in place:

- ☐ ARM GNU Toolchain 14.2 installed; arm-none-eabi-gcc available at C:/Users/rusle/.mcuxpressotools/arm-gnu-toolchain-14.2.rel1-mingw-w64-x86_64-arm-none-eabi (or update TOOLCHAIN in Makefile)
- ☐ NXP MCUXpresso SDK extracted; update SDK_ROOT in Makefile if needed
- ☐ pyocd available: C:/Development/RTOS/.venv/Scripts/pyocd.exe (local venv; NXP.MCXXN236_DFP pack pre-installed)
- ☐ FRDM-MCXXN236 connected via the **MCU-LINK USB** port
- ☐ Python 3 + pyserial for serial monitoring (pip3 install pyserial)

3. Project Structure

```
lab01_setup_verify/
Makefile                <- build, flash, debug targets
src/
  main.c                <- task definitions, LED init, scheduler
  start
  FreeRTOSConfig.h      <- kernel configuration
<SDK_ROOT>/             <- board support, FreeRTOS kernel, drivers (
  external)
  rtos/freertos/freertos-kernel-upstream/ <- FreeRTOS kernel
  devices/MCX/MCXXN/MCXXN236/             <- device startup + linker
  script
  examples/_boards/frdm-mcxcn236/         <- board.h, clock_config,
  pin_mux
```

The build pulls all SDK sources directly from SDK_ROOT in the Makefile — there is nothing to clone or install separately.

3.1. What the code does

Two FreeRTOS tasks run continuously after the scheduler starts:

Task	Priority	Period	What it does
vBlinkTask	2 (HIGH)	500 ms	Toggles the physical red LED (PIO4_18) and prints LED ON/OFF
vHeartbeatTask	1 (LOW)	1000 ms	Prints the FreeRTOS tick uptime in milliseconds

A shared **UART mutex** (xUartMutex) prevents both tasks from calling PRINTF simultaneously — the simplest possible example of a critical section.

Tip

Hardware note: The red LED is active-low on GPIO4 pin 18 (board pad N10). LOGIC_LED_ON = 0 (GPIO low → LED on), LOGIC_LED_OFF = 1 (GPIO high → LED off). LED_RED_TOGGLE() is a macro defined in the SDK's board.h.

4. Part A — Build and Flash

4.1. Step 1 — Build

```
cd labs/lab01_setup_verify
mingw32-make
```

Expected output (sizes will vary slightly):

```
arm-none-eabi-gcc ... -o lab01.elf
arm-none-eabi-objcopy -O binary lab01.elf lab01.bin
arm-none-eabi-size lab01.elf
   text    data     bss     dec      hex filename
  31588     20    16060   47668   ba34 lab01.elf
```

Note

Toolchain not found: Edit the TOOLCHAIN variable at the top of Makefile to point to your arm-none-eabi-gcc installation directory.

SDK not found: Edit SDK_ROOT in Makefile to match where you extracted the NXP MCUXpresso SDK.

4.2. Step 2 — Flash

Connect the board via the MCU-LINK USB port, then:

```
mingw32-make flash
```

This uses pyocd to erase, program, and reset the MCU. The progress bar completes and the board starts running immediately after the reset.

5. Part B — Verify: UART Output and LED

5.1. Serial terminal

The board exposes a USB-CDC virtual COM port over the MCU-LINK USB cable.

Find the port:

```
# Windows: check Device Manager for COMx port
# Example output: COM3
```

Monitor with screen:

```
screen COMx 115200
# Press Ctrl-A then Ctrl-\ to quit
```

Or with Python (clean, non-garbled capture):

```
python3 -c "
import serial, time, sys
s = serial.Serial('COMx', 115200, timeout=0.1)
s.reset_input_buffer()
end = time.time() + 10
while time.time() < end:
    d = s.read(256)
    if d: sys.stdout.buffer.write(d); sys.stdout.buffer.flush()
s.close()
"
```

Windows: PuTTY or Tera Term — COMx, 115200 baud, 8N1.

5.2. Expected output

```
=== RTOS Lab 01: Toolchain & Environment Verify ===
SDK: BOARD_InitHardware + PRINTF | CPU: PLL 150 MHz
Tasks: Blink (500 ms) | Heartbeat (1000 ms)

[MAIN] Starting scheduler...

[BLINK] LED ON   (toggle #1)
[BLINK] LED OFF  (toggle #2)
[HB]    uptime = 1003 ms
[BLINK] LED ON   (toggle #3)
[BLINK] LED OFF  (toggle #4)
[HB]    uptime = 2007 ms
```

💡 Tip

What to observe:

- [BLINK] prints *twice* for every one [HB] line, reflecting 500 ms vs. 1000 ms periods.
- Uptime increments by approximately 1000 ms per [HB] line.
- The **red LED** on the board blinks visibly at 2 Hz (500 ms on, 500 ms off).
- Output never garbles — the mutex serialises both tasks' PRINTF calls.

✓ Checkpoint

Capture at least 5 seconds of terminal output **and** photograph the blinking LED.

6. Part C — Experiments

Work through these in order. Record UART output for each one.

6.1. Experiment 1 — Priority swap

In main.c, swap the task priorities:

```
1 xTaskCreate(vBlinkTask,      "Blink", 512, NULL, 1, NULL); /* was 2
   */
2 xTaskCreate(vHeartbeatTask, "HB",    512, NULL, 2, NULL); /* was 1
   */
```

Rebuild (make), flash, and observe the serial output.

? Question

Does the output order or timing change? Why or why not?

Hint: both tasks spend almost all their time blocked inside `vTaskDelay`. Think about what happens when both unblock at the exact same tick.

6.2. Experiment 2 — Equal priorities

Set both tasks to the same priority:

```
1 xTaskCreate(vBlinkTask, "Blink", 512, NULL, 1, NULL);
2 xTaskCreate(vHeartbeatTask, "HB", 512, NULL, 1, NULL);
```

? Question

What scheduling policy does FreeRTOS apply when two tasks have equal priority and are both ready? Where in `FreeRTOSConfig.h` is this policy enabled?

6.3. Experiment 3 — Periods and CPU load

Restore original priorities (Blink = 2, HB = 1). Change `vBlinkTask`'s delay from 500 ms to **50 ms**:

```
1 vTaskDelay(pdMS_TO_TICKS(50));
```

Rebuild and flash. The red LED should now blink at 10 Hz.

? Question

Using the utilization formula $U = C/T$, estimate the CPU load of `vBlinkTask`. C is the time spent executing (very short — one `PRINTF` call); $T = 50$ ms.

Does the scheduler still meet the 1000 ms heartbeat deadline? Why?

6.4. Experiment 4 — Remove the mutex (observe the race)

Keep the 50 ms Blink period from Experiment 3. In both task bodies, comment out the `xSemaphoreTake` / `xSemaphoreGive` lines around `PRINTF`:

```
1 /* xSemaphoreTake(xUartMutex, portMAX_DELAY); */
2 PRINTF("[BLINK] LED %s (toggle #%u)\r\n", state, (unsigned)count);
3 /* xSemaphoreGive(xUartMutex); */
```

? Question

Do you observe garbled or interleaved output on the terminal?

Why is this a **race condition** even on a *single-core* MCU? What property of a mutex prevents it?

! Note

Restore the mutex calls before submitting.

7. Deliverables

Submit the following to the course LMS by the posted deadline:

#	Item	Details
1	Terminal capture	5+ seconds of correct [BLINK] / [HB] output
2	LED photo	Board with red LED visibly on or blinking
3	Answers to Q1–Q4	A few sentences each; reference the code or output
4	Modified <code>main.c</code>	From Experiment 3 (50 ms Blink, mutex restored)

8. Key Concepts Introduced

Concept	Where you saw it
Preemptive, priority-based scheduling	Priority swap (Experiment 1)
Time-slicing among equal-priority tasks	Equal priority (Experiment 2)
CPU utilization $U = C/T$	Short-period Blink (Experiment 3)
Race condition on a shared resource	Mutex removal (Experiment 4)
Mutex as a critical-section guard	<code>xUartMutex</code> throughout
Periodic task with <code>vTaskDelay</code>	<code>vBlinkTask</code> , <code>vHeartbeatTask</code>

These are the building blocks for Module 2 (scheduling theory) and Module 3 (synchronisation).

9. Implementation Notes

9.1. Why `configRUN_FREERTOS_SECURE_ONLY 1`

The MCMXN236 boots and runs entirely in the ARM **secure world**. The FreeRTOS ARM_CM33_NTZ port must know this to set the correct `EXC_RETURN` value when launching the first task:

Config	EXC_RETURN	Bit[6]	Result
<code>configRUN_FREERTOS_SECURE_ONLY 0</code> (default)	0xFFFFFBC	0 – non-secure	Immediate fault: CPU already in secure mode
<code>configRUN_FREERTOS_SECURE_ONLY 1</code>	0xFFFFFBD	1 – secure	First task starts correctly

Without this setting the scheduler appears to start (the banner prints) but no tasks ever execute — the symptom is total silence after [MAIN] Starting scheduler...

9.2. Why the SDK FreeRTOS kernel

This lab uses the NXP SDK's `freertos-kernel-upstream` (inside `SDK_ROOT/rtos/freertos/`) rather than a standalone clone. The SDK kernel is the version NXP has tested against this chip and BSP. A development-branch clone may have incompatible `portasm.c` changes that interact badly with the secure-world boot sequence.

9.3. LED hardware

LED	GPIO	Pin	Board pad	Active level
Red	GPIO4	18	N10	Low (0 = ON)
Blue	GPIO4	17	R9	Low (0 = ON)
Green	GPIO4	19	R10	Low (0 = ON)

All three share GPIO4. The pin mux is set to `kPORT_MuxAlt0` (GPIO function) in `main.c` before the scheduler starts, using `PORT_SetPinConfig` and `GPIO_PinInit`.

10. Reference

Resource	Relevant sections
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 1 (tasks), Ch. 3 (queue/semaphore basics)
Laplante, <i>Real-Time Systems Design</i>	Ch. 1–2 (task model, deadline classes)
FreeRTOS documentation	<code>xTaskCreate</code> , <code>vTaskDelay</code> , <code>xSemaphoreCreateMutex</code>
NXP AN13036 SDK <code>board.h</code>	FreeRTOS on Cortex-M33 with TrustZone <code>LED_RED_TOGGLE</code> , <code>LOGIC_LED_ON/OFF</code> macros